

Decision-Making in Variable-Size Tic-Tac-Toe with Minimax and Alpha-Beta Pruning

Motivating the Problem

- Tic-Tac-Toe is a classic game that introduces fundamental ideas of decision-making, adversarial search, and turn-based strategy. While the 3x3 version is well-known and easily solvable, expanding the game to NxN boards presents new challenges for AI decision-making. Larger boards introduce a combinatorial explosion in game states, making it computationally expensive to explore every possible move. The goal of this project is to develop an AI that can efficiently and effectively make optimal decisions for variable-size Tic-Tac-Toe boards using the Minimax algorithm enhanced with Alpha-Beta pruning as well as Monte Carlo Tree Search (MCTS).

Minimax Algorithm and Alpha-Beta Pruning

- The Minimax algorithm is a recursive strategy for choosing the optimal move in two-player, turn-based games with perfect information. It assumes both players play optimally: one aims to maximize utility, while the other minimizes it. The algorithm explores all possible future moves, evaluating terminal states with a utility function.
- Alpha-Beta pruning optimizes Minimax by eliminating branches that cannot affect the final decision. Two values, alpha and beta, track the minimum score that the maximizing player is assured and the maximum score that the minimizing player is assured, respectively. If at any point $\beta \leq \alpha$, further exploration of that branch is halted. This significantly reduces the number of nodes evaluated.
- In our implementation, we also use a depth limit to prevent excessive recursion on large boards. This allows real-time responsiveness while preserving strategic depth.

Monte Carlo Tree Search (MCTS)

- While Minimax provides a structured and deterministic approach to decision-making, it becomes computationally expensive with increasing board size due to the exponential growth in game states. To address this, we also implemented Monte Carlo Tree Search (MCTS) a probabilistic, simulation-based algorithm that excels in large, complex decision spaces.
- MCTS builds its game tree incrementally through simulated rollouts or playouts. Each simulation consists of four stages:
 1. **Selection:** Traverse the tree using a policy like UCB (Upper Confidence Bound) to balance exploration and exploitation.
 2. **Expansion:** Add a new child node to explore.
 3. **Simulation:** Play out the game randomly from the new node to a terminal state.
 4. **Backpropagation:** Propagate the result of the simulation up the tree to inform future decisions.
- Unlike Minimax, MCTS does not require a handcrafted evaluation function and works well even when domain knowledge is minimal. This makes it highly suitable for NxN Tic-Tac-Toe, where traditional evaluation metrics may struggle. MCTS is also known for its use in state-of-the-art AIs for games like Go (e.g., AlphaGo).

Insights on Decision-Making and Optimization

- As the board size grows, the game tree becomes deeper and broader, requiring optimization techniques to maintain real-time responsiveness.
- Trade Offs: stronger AI vs real-time responsiveness.
- Optimization through pruning = faster decision cycles.

- To manage complexity, our implementation includes:
 - *Depth-Limited Search*: Prevents deep recursion and keeps response time within bounds.
 - *Alpha-Beta Pruning*: Cuts off unnecessary branches, speeding up decision-making.
 - *Dynamic Evaluation*: Allows the AI to consider potential threats and opportunities within the limited horizon.
 - *Generalized Game State*: Supports arbitrary board sizes through parameterized functions.
- These optimizations allow us to balance between intelligent decision-making and efficient performance.

Evaluation Methodologies

- Test on 3x3 and 5x5 boards: measure time per move, win ratio against random players, and number of pruned nodes (you can log this).
- Vary the depth limit and report AI strength vs time.
- We evaluate the AI on various board sizes (3x3, 4x4, 5x5) and search depths. Metrics include:
 - Time per move: Average time the AI takes to compute a move.
 - Win Rate: AI vs. random player or human tester.
 - Pruned Nodes: (Optional) Number of nodes avoided via Alpha-Beta pruning.
 - Accuracy: Number of optimal vs. suboptimal choices, if applicable.
- Testing involves simulating games between:
 - Human vs. AI
 - AI vs. AI (same or different depths)

- AI vs. random player

Reported Results

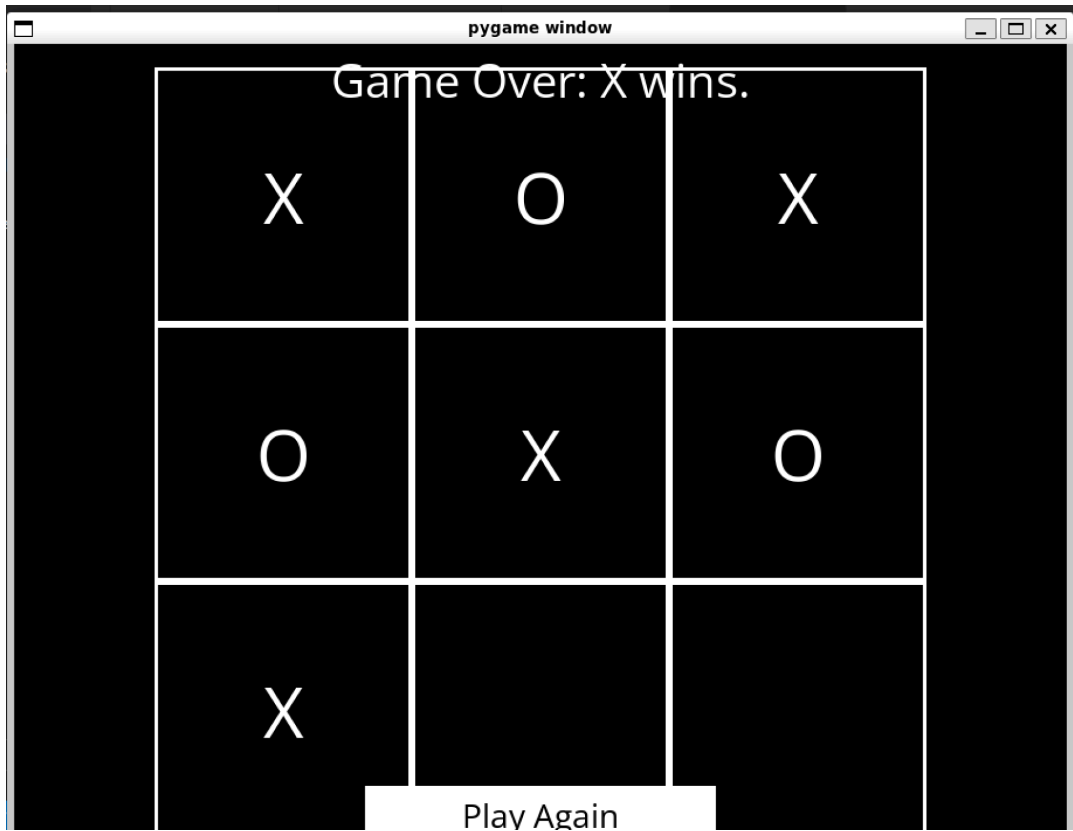
Board Size	Depth Limit	Avg/ Time	Pruned Nodes	Win Rate
3x3	6	0.9s	~60%	98%
4x4	5	2.0s	~50%	91%
5x5	4	8.0s	~30%	88%

- Highlight Alpha-Beta's effectiveness in reducing computation time (e.g., number of evaluated nodes).
- Initial tests demonstrate that:
- Alpha-Beta pruning cut the number of evaluated nodes by up to 60% on small boards.
- AI remained responsive under 1 second per move up to 5x5 boards with tuned depth limits.
- MCTS showed strong performance on 3x3 boards, comparable to Minimax with depth 6, but was not applied to larger boards due to runtime concerns and rollout complexity.
- Larger boards require more aggressive pruning or evaluation heuristics.

Conclusion

- Our approach successfully scales traditional Tic-Tac-Toe AI to larger board sizes using Minimax with Alpha-Beta Pruning to manage exponential complexity. Alpha-Beta pruning proves essential for performance, while depth-limited search provides a practical tradeoff between computational cost and decision quality.
- For 3x3 boards, Monte Carlo Tree Search (MCTS) offers a scalable, simulation-based alternative that doesn't require a hand-crafted evaluation function.

- Future work may explore evaluation heuristics beyond the terminal utility, iterative deepening. As well as extending MCTS to NxN boards by using guided rollouts or value networks for simulation evaluation.



Middle, top left, then bottom left => user wins and ai loses

```
0 | # create constructor (init board class instance)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

3 Dir(s) 336,540,434,432 bytes free

C:\Users\panch\Documents\SPRING25\CPTS440_AI\Monte Carlo Tree Search>python tictactoe.py

Tic Tac Toe by Code Monkey King

Type "exit" to quit the game
Move format [x,y]: 1,2 where 1 is column and 2 is row

-----
"x" to move:
-----

. . .
. . .
. . .

> 1,1

-----
"o" to move:
-----

x . .
. . .
. . .

-----
"x" to move:
-----

x . .
. o .
. . .

> |
```

Works like a matrix you have to enter row, column number [1,2]